Git Basics Assignment 1

1. Repository setup

```
→  git_tasks git init
Initialized empty Git repository in /Users/ankushhh28/git_tasks/.git/
→  git_tasks git:(main)
→  git_tasks git:(main) git config --global user.name "ankushh2004"
→  git_tasks git:(main) git config --global user.email "asanodiya@ancilar.com"
→  git_tasks git:(main) git config --list

→  git_tasks git:(main) git config --list
```

```
credential.helper=osxkeychain
init.defaultbranch=main
user.name=ankushh2004
user.email=asanodiya@ancilar.com
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
core.ignorecase=true
core.precomposeunicode=true
remote.origin.url=https://github.com/ankushh2004/git_demo.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
(END)
```

```
→  git_tasks git:(main) git remote add origin https://github.com/ankushh2004/git_demo.git
→  git_tasks git:(main) git remote -v
origin  https://github.com/ankushh2004/git_demo.git (fetch)
origin  https://github.com/ankushh2004/git_demo.git (push)
→  git_tasks git:(main)
```

Query:Why is Git considered one of the most popular version control systems in the industry?

- Before Git m, version control systems were used by developers to manage their code. They were called SCCS (Source Code Control System).
- SCCS was a proprietary software that was used to manage the history of code.
- It was expensive and not very user-friendly.
- Git was created to replace SCCS and to make version control more accessible and user-friendly as it is open-source and free to all.
- Some common version control systems are Subversion (SVN), CVS, and Perforce.

2. Working with files and commits

```
➜  git_tasks git:(main) ls
➜  git_tasks git:(main) touch demo.txt
➜  git_tasks git:(main) ✗ ls
demo.txt
➜  git_tasks git:(main) ✗ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        demo.txt

nothing added to commit but untracked files present (use "git add" to track)
➜  git_tasks git:(main) ✗ git add demo.txt
➜  git_tasks git:(main) ✗ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   demo.txt

➜  git_tasks git:(main) ✗ git commmit -m "Adding new file demo"
git: 'commmit' is not a git command. See 'git --help'.

The most similar command is
        commit
➜  git_tasks git:(main) ✗ git commmit -m"Adding new file demo"
git: 'commmit' is not a git command. See 'git --help'.

The most similar command is
        commit
➜  git_tasks git:(main) ✗ git commit -m"Adding new file demo"
[main (root-commit) 5ec5412] Adding new file demo
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 demo.txt
➜  git_tasks git:(main) git status
On branch main
nothing to commit, working tree clean
➜  git_tasks git:(main)
```

```
➜  git_tasks git:(main) echo "This is the demo file" > demo.txt
➜  git_tasks git:(main) ✗ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   demo.txt

no changes added to commit (use "git add" and/or "git commit -a")
➜  git_tasks git:(main) ✗
```

```
➜  git_tasks git:(main) ✗ git add .
➜  git_tasks git:(main) ✗ git commit -m"Add content to the demo file"
[main a5b2334] Add content to the demo file
 1 file changed, 1 insertion(+)
➜  git_tasks git:(main) git status
On branch main
nothing to commit, working tree clean
➜  git_tasks git:(main) git log
```

```
commit a5b233449dfb7b9e22526ffce65981471ed5dae9 (HEAD -> main)
Author: ankushh2004 <asanodiya@ancilar.com>
Date:   Wed Oct 1 15:26:39 2025 +0530

    Add content to the demo file

commit 5ec54127e8013bc3f84248f26a1a4b5368bcdf58
Author: ankushh2004 <asanodiya@ancilar.com>
Date:   Wed Oct 1 15:15:16 2025 +0530

    Adding new file demo
(END)
```

```
➜  git_tasks git:(main) git log
➜  git_tasks git:(main) git log --oneline
➜  git_tasks git:(main)
```

```
a5b2334 (HEAD -> main) Add content to the demo file
5ec5412 Adding new file demo
(END)
```

```
git_tasks git:(main) git log --oneline
git_tasks git:(main) git log --graph
git_tasks git:(main)
```

```
* commit a5b233449dfb7b9e22526ffce65981471ed5dae9 (HEAD -> main)
| Author: ankushh2004 <asanodiya@ancilar.com>
| Date:   Wed Oct 1 15:26:39 2025 +0530
|
|     Add content to the demo file
|
* commit 5ec54127e8013bc3f84248f26a1a4b5368bcdf58
  Author: ankushh2004 <asanodiya@ancilar.com>
  Date:   Wed Oct 1 15:15:16 2025 +0530

      Adding new file demo
(END)
```

Query:What is a SHA hash in Git, and why is it important?

- SHA stands for Secure Hash Algorithm.
- Basically, the SHA algorithm is used for generating a unique commit ID.
- Every commit has a unique hash ID which consists of a 40 characters alpha-numeric unique ID to differentiate commits .

3. Understanding remote operations

```
git_tasks git:(main) git clone https://github.com/ankushhh28/Mock-Ninja.git
Cloning into 'Mock-Ninja'...
remote: Enumerating objects: 9029, done.
remote: Total 9029 (delta 0), reused 0 (delta 0), pack-reused 9029 (from 2)
Receiving objects: 100% (9029/9029), 86.15 MiB | 12.49 MiB/s, done.
Resolving deltas: 100% (2883/2883), done.
git_tasks git:(main) ✗
```

```
git-tasks git:(main) git fetch
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1024 bytes | 204.00 KiB/s, done.
From github.com:ankushh2004/new-repo
   4b49b46..08e5edc  main       -> origin/main
git-tasks git:(main) git log --oneline
git-tasks git:(main) ls
test1.txt
git-tasks git:(main)
```

```
4b49b46 (HEAD -> main) Perform push operation for git 1 assignment
59edccd Revert "test1.txt also contains fifth line in file"
aa0e664 fourth line added
0a75c81 Revert "my second commit"
f3a4786 my second commit
7dfb4b5 first line added
(END)
```

```
➜  git-tasks git:(main) git pull
There is no tracking information for the current branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.

    git pull <remote> <branch>

If you wish to set tracking information for this branch you can do so with:

    git branch --set-upstream-to=origin/<branch> main

➜  git-tasks git:(main) git pull origin main
'From github.com:ankushh2004/new-repo
 * branch            main       -> FETCH_HEAD
Updating 4b49b46..08e5edc
Fast-forward
 assignment-1-test.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 assignment-1-test.txt
➜  git-tasks git:(main) ls
assignment-1-test.txt test1.txt
➜  git-tasks git:(main) git log --oneline -2
➜  git-tasks git:(main)
```

```
08e5edc (HEAD -> main, origin/main, origin/HEAD) Create assignment-1-test.txt
4b49b46 Perform push operation for git 1 assignment
(END)
```

```
➜  git-tasks git:(main) ✗ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test1.txt

no changes added to commit (use "git add" and/or "git commit -a")
➜  git-tasks git:(main) ✗ git add .
➜  git-tasks git:(main) ✗ git commit -m"Perform push operation for git 1 assignment"
[main 4b49b46] Perform push operation for git 1 assignment
 1 file changed, 1 insertion(+)
➜  git-tasks git:(main) git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 307 bytes | 307.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:ankushh2004/new-repo.git
   59edccd..4b49b46  main -> main
➜  git-tasks git:(main)
```

Query: What is the key difference between git fetch and git pull?

- **git fetch**
    1. Before `git fetch`, developers had to manually check or pull all changes from a remote repository. The problem was that `git pull` would automatically merge changes, which could lead to unwanted updates or merge conflicts if you were not ready. To solve this, Git introduced `git fetch`.
    2. git fetch is a command that retrieves updates from a remote repository but does not merge them with your local branch. It is like downloading the latest changes without applying them directly to your working directory.
    3. The command fetches all the references (branches, tags, etc.) from the remote, allowing you to review or integrate those updates manually when you are ready.
    4. When we run git fetch, Git connects to the remote repository (typically called origin), downloads any new commits or branches, and stores them in your local repository under remote-tracking branches.
    5. These remote-tracking branches (like origin/main) represent the latest state of the branches in the remote repository.
    6. To save the changes to our local repository or our workspace we need to use the “git fetch” command.
- **git pull**
    1. Git pull is a command which is used to fetch and integrate the changes which are present in the remote repository to the local repository.

2. Git pull is basically a combination of git merge and git fetch which is used to update the local branch with the changes available in the remote repository branch.

4. Commit Lifecycle & modifications

```
➜  demo git:(main) git status
On branch main
nothing to commit, working tree clean
➜  demo git:(main) echo "Demo file  is used for testing">> demo.txt
➜  demo git:(main) ✗ git add demo.txt
➜  demo git:(main) ✗ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   demo.txt

➜  demo git:(main) ✗ git restore --staged demo.txt
➜  demo git:(main) ✗ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   demo.txt

no changes added to commit (use "git add" and/or "git commit -a")
➜  demo git:(main) ✗ ~
```

```
➜  demo git:(main) git rm demo.txt
rm 'demo.txt'
➜  demo git:(main) ✗ ls
➜  demo git:(main) ✗ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    demo.txt

➜  demo git:(main) ✗ git commit -m"demo.txt deleted successfully!"
dquote>
➜  demo git:(main) ✗ git add .
➜  demo git:(main) ✗ git commit -m"demo.txt deleted successfully
dquote>
➜  demo git:(main) ✗ git commit -m"demo.txt deleted successfully"
[main a1e6d55] demo.txt deleted successfully
 1 file changed, 1 deletion(-)
 delete mode 100644 demo.txt
➜  demo git:(main) git log --oneline
➜  demo git:(main)
```

```
a1e6d55 (HEAD -> main) demo.txt deleted successfully
3bec8ff Adding demo.txt file
dcf0d93 Add content to the demo file
fe164d6 Add demo  file
(END)
```

```
➜  demo git:(main) git status
On branch main
nothing to commit, working tree clean
➜  demo git:(main) ls
➜  demo git:(main) echo "This is the test file" > test.txt
➜  demo git:(main) ✗ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test.txt

nothing added to commit but untracked files present (use "git add" to track)
➜  demo git:(main) ✗ git add .
➜  demo git:(main) ✗ git commit -m "Add test file succesfully!"
dquote>
➜  demo git:(main) ✗ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   test.txt

➜  demo git:(main) ✗ git commit -m"test file added successfully!"
dquote>
➜  demo git:(main) ✗ git commit -m "test file added successfully
dquote>
➜  demo git:(main) ✗ git commit -m "test file added successfully"
[main 0052976] test file added successfully
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
➜  demo git:(main) git log --oneline
'q
➜  demo git:(main) git commit --ammend -m "Commit changed successfully!"
dquote> git log --oneline
➜  demo git:(main) git commit --ammend -m"Commit changed successfully
dquote>
➜  demo git:(main) git commit --ammend -m"Commit changed successfully"
```

```
                        use specified template file
    -e, --edit          force edit of commit
    --cleanup <mode>    how to strip spaces and #comments from message
    --status            include status in commit message template
    -S, --gpg-sign[=<key-id>]
                        GPG sign commit

Commit contents options
    -a, --all           commit all changed files
    -i, --include       add specified files to index for commit
    --interactive       interactively add files
    -p, --patch         interactively add changes
    -o, --only          commit only specified files
    -n, --no-verify     bypass pre-commit and commit-msg hooks
    --dry-run           show what would be committed
    --short             show status concisely
    --branch            show branch information
    --ahead-behind      compute full ahead/behind values
    --porcelain         machine-readable output
    --long              show status in long format (default)
    -z, --null          terminate entries with NUL
    --amend             amend previous commit
    --no-post-rewrite   bypass post-rewrite hook
    -u, --untracked-files[=<mode>]
                        show untracked files, optional modes: all, normal, no. (Default: all)
    --pathspec-from-file <file>
                        read pathspec from file
    --pathspec-file-nul   with --pathspec-from-file, pathspec elements are separated with NUL character

➜  demo git:(main) git commit --amend -m"Commit changed successfully"
[main 2f61249] Commit changed successfully
 Date: Wed Oct 1 17:36:20 2025 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
➜  demo git:(main) git log --oneline
➜  demo git:(main)
```

```
2f61249 (HEAD -> main) Commit changed successfully
a1e6d55 demo.txt deleted successfully
3bec8ff Adding demo.txt file
dcf0d93 Add content to the demo file
fe164d6 Add demo  file
(END)
```

Query: How does `git reset` differ from `git rm` ?

- **git reset**
    1. git reset is primarily used to undo changes, modify the commit history, or unstage changes.
    2. To undo local commits you haven't pushed yet.
    3. To unstage specific files.

- **git rm**
    1. git rm is used to remove files from the Git repository.
    2. By default, git rm removes the file from both the staging area (index) and the working directory.

5. Advanced Operations

```
➜  demo git:(main) ls
test.txt
➜  demo git:(main) cat test.txt
This is the test file
➜  demo git:(main) echo "created by Ankush" >> test.txt
➜  demo git:(main) ✗ cat test.txt
This is the test file
created by Ankush
➜  demo git:(main) ✗ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.txt

no changes added to commit (use "git add" and/or "git commit -a")
➜  demo git:(main) ✗ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.txt

no changes added to commit (use "git add" and/or "git commit -a")
➜  demo git:(main) ✗ git stash
Saved working directory and index state WIP on main: 2f61249 Commit changed successfully
➜  demo git:(main) git status
On branch main
nothing to commit, working tree clean
➜  demo git:(main) git stash pop
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.txt
```

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (f4aefdcfca1265bf7e5d63bc99c9f7e35a64902b)
➜  demo git:(main) ✗ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.txt

no changes added to commit (use "git add" and/or "git commit -a")
➜  demo git:(main) ✗ git commit -m"Add more content on test file"
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.txt

no changes added to commit (use "git add" and/or "git commit -a")
➜  demo git:(main) ✗ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.txt

no changes added to commit (use "git add" and/or "git commit -a")
➜  demo git:(main) ✗ git add .
➜  demo git:(main) ✗ git commit -m"Add more content on test file"
[main d5be905] Add more content on test file
 1 file changed, 1 insertion(+)
➜  demo git:(main)
```

```
➜  demo git:(main) ✗ touch one.txt
➜  demo git:(main) ✗ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   test.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        one.txt

➜  demo git:(main) ✗ git add .
➜  demo git:(main) ✗ git commit-m"one.txt created"
git: 'commit-mone.txt created' is not a git command. See 'git --help'.
➜  demo git:(main) ✗ git commit -m"one.txt created"
[main 47affad] one.txt created
 2 files changed, 1 deletion(-)
 create mode 100644 one.txt
➜  demo git:(main) git status
On branch main
nothing to commit, working tree clean
➜  demo git:(main) git log
➜  demo git:(main) git revert 47affadfa2808fe683a4ff27f42c4403bcaeea2a
[main 9a80ab4] Revert "one.txt created successfully!" This reverts commit 47affadfa2808fe683a4ff27f42c4403bcaeea2a.
 2 files changed, 1 insertion(+)
 delete mode 100644 one.txt
➜  demo git:(main) git log
➜  demo git:(main)
```

Query: How does Git calculate file differences when using commands like git diff? (Hint: HEAD)

=>git diff works , like it compares the screenshot , it stores complete file , snapshots as blob , and when hashes differ , it uses myers diffing algorithm to show difference